



1.2. Minimum Cost Spanning Tree

Main Section:

1. Greedy Approach

Contents:

1. Problem Definition

2. Kruskal's Algorithm

Applying Kruskal's Algorithm to the Graph

Time Complexity of Kruskal's Algorithm

Advantages of Kruskal's Algorithm

Disadvantages of Kruskal's Algorithm

3. Prim's Algorithm

The Core Idea of Prim's Algorithm

Applying Prim's Algorithm to the Graph

Time Complexity of Prim's Algorithm

Advantages of Prim's Algorithm

Disadvantages of Prim's Algorithm

Summary

References

The Minimum Cost Spanning Tree is another one of those classic Computer Science problems that appears everywhere once you start studying algorithms, networks, and optimization. But unlike the Knapsack Problem—which focuses

on selecting the “best” objects under limited capacity—the Minimum Spanning Tree problem is about something completely different: Connection.

More specifically: ***“How do we connect everything together... using the minimum possible cost?”***

And honestly, this problem is much more practical than most beginners realize. Imagine building roads between cities, connecting computers in a network, designing electrical systems, creating internet cable infrastructure, setting up water pipelines. In all of these situations, one goal always appears: ***Connect every location together while spending the least amount of money possible.***

And that is exactly the purpose of the Minimum Spanning Tree. Now at first, the problem may look intimidating because it involves graphs, vertices, edges, and cycles. But underneath all the terminology, the idea is surprisingly intuitive. We are simply trying to build the cheapest possible network that still keeps everything connected. No unnecessary paths. No wasted cost. No redundant loops.

And just like the Knapsack Problem, Greedy Algorithms play a major role here. In fact, some of the most famous algorithms in Computer Science—Kruskal's Algorithm and Prim's Algorithm—were specifically designed to solve this exact problem.

So before we explore those algorithms, we first need to properly understand one very important concept: ***What exactly is a spanning tree?***

1. Problem Definition

Before understanding the Minimum Spanning Tree, we first need to understand two smaller ideas: Graphs and Trees. You have probably heard these terms before, even if you have not studied them deeply yet. And if not, don't worry—we will study trees and graphs more formally in future sections.

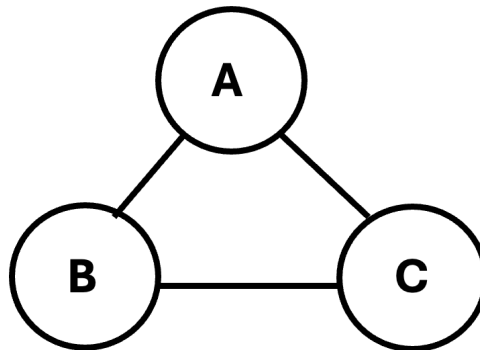
For now, think of a graph as a structure made of: Vertices (nodes) and edges (connections between nodes).

A graph can represent many real-world systems: Cities connected by roads; People connected through social networks; Computers connected through cables; Airports connected through flights;

The important thing is this: Graphs are flexible. They can contain many different paths, and sometimes, those paths form cycles.

A cycle happens when you can start from one node, travel through several edges, and eventually return back to the same node again.

For example:

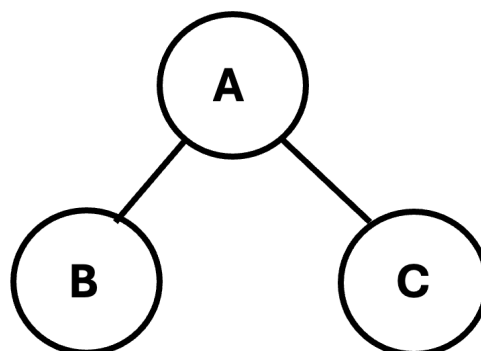


This is a graph: $A \rightarrow B \rightarrow C \rightarrow A$

This creates a loop. Now a tree is actually a very special type of graph.

A tree still has vertices and edges, but it has one important restriction: It contains no cycles. Meaning if you start from any node and continue traveling through the connections, you will never come back to the same node again using a different path.

For example:



This is a graph: $A \rightarrow B$ and $A \rightarrow C$

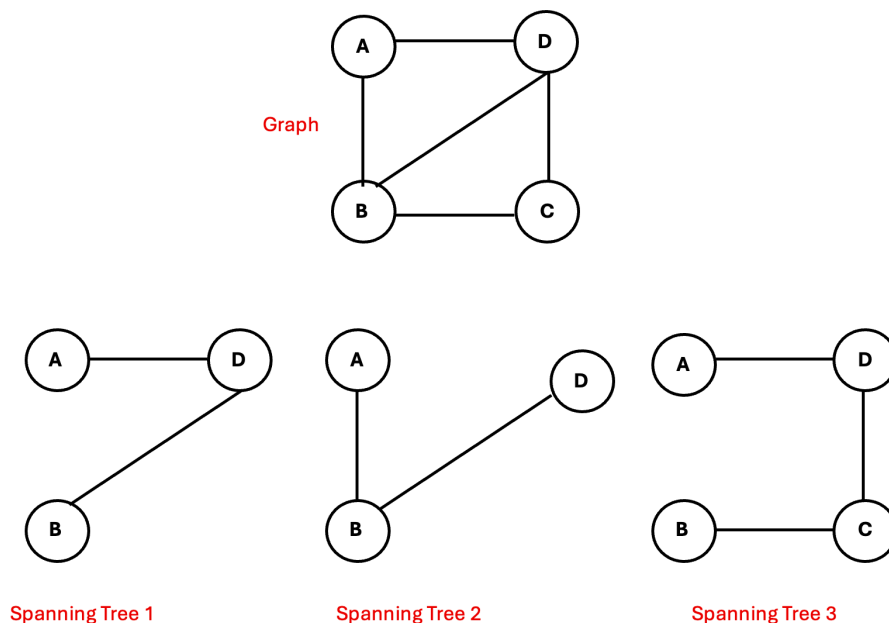
In other words: A tree is a fully connected structure without loops. And this restriction is extremely important because cycles often create redundancy.

Imagine building roads between cities. If two cities are already connected indirectly, adding another unnecessary loop may increase cost without

improving connectivity very much. So trees help us remove unnecessary connections while still keeping everything reachable.

Now let's go one step further. Suppose we already have a graph. Inside that graph, there may exist multiple different ways to connect all vertices without creating cycles. Each possible version is called a spanning tree.

A spanning tree is simply: A tree created from a graph that connects all vertices together without cycles.



And here's something interesting: One graph can generate many possible spanning trees. For example, imagine a graph containing 5 vertices and 4 edges connecting them. Different combinations of edges can still connect all 5 vertices while avoiding cycles.

So naturally, a question appears: ***"How many spanning trees can a graph actually have?"***. At first, you might think: ***"Just choose combinations of edges."***

→ For example: 5C_4 . But there's a problem. Not every combination produces a valid tree. Some combinations create cycles.

And since trees are not allowed to contain cycles, those invalid combinations must be removed.

That's why we subtract combinations that form loops. The actual number of spanning trees is ${}^5C_4 - \text{number of cycles}$.

So the problem is not just about choosing edges randomly. It is about choosing the correct edges that connect every vertex, avoid cycles, and maintain a valid tree structure.

Now we finally arrive at the most important concept: The Minimum Spanning Tree (MST). A Minimum Spanning Tree is simply a spanning tree with the smallest possible total weight.

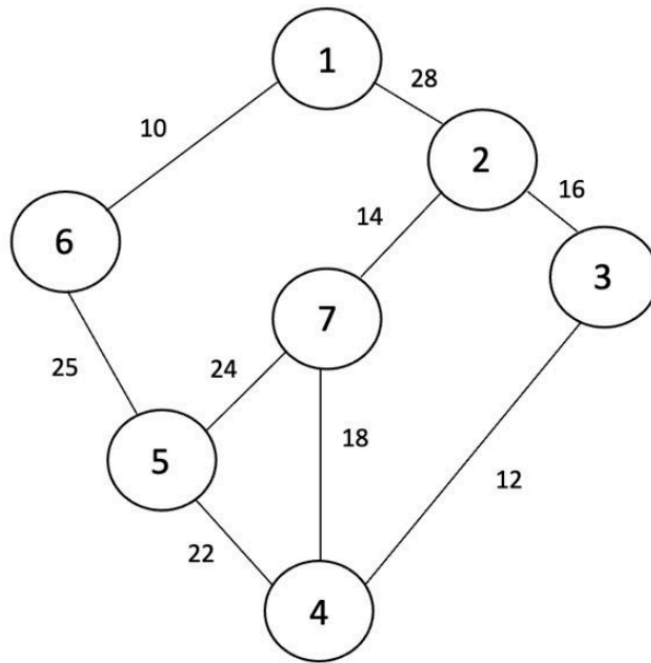
In weighted graphs, each edge has a cost or weight attached to it. This weight could represent distance, construction cost, travel time, cable length, network latency. The MST chooses the connections whose total cost is minimum while still connecting every vertex together.

So among all possible spanning trees: We select the one with the smallest total weight. Mathematically, the MST minimizes: $\sum w(e)$, where: $w(e)$ represents the weight of each selected edge.

And this is where Greedy Algorithms become extremely powerful. Instead of testing every possible spanning tree—which would become impossibly expensive for large graphs—we make locally optimal decisions step by step.

Two famous algorithms do exactly this: Kruskal's Algorithm and Prim's Algorithm. Both use the Greedy approach. Both build the Minimum Spanning Tree incrementally. And both demonstrate one of the most important ideas in algorithm design: ***Sometimes, repeatedly making the best local decision, eventually leads to the globally optimal solution.***

Consider the following graph:



We will use this graph for the following two contents.

2. Kruskal's Algorithm

Kruskal's Algorithm is a greedy algorithm introduced by Joseph Kruskal to construct a **Minimum Spanning Tree (MST)**. And just like every greedy algorithm, it follows one simple philosophy:

▮ ***"Choose the best possible option at the current moment."***

In this case, the "best" option simply means: ***Choose the edge with the smallest weight available.***

So instead of focusing on vertices first... Kruskal's Algorithm focuses entirely on **edges**. The process is surprisingly straightforward:

1. Sort all edges in ascending order based on weight.
2. Select the smallest edge.
3. Keep adding the next smallest edge, but only if adding that edge does **not** create a cycle.

And that last rule is the most important part.

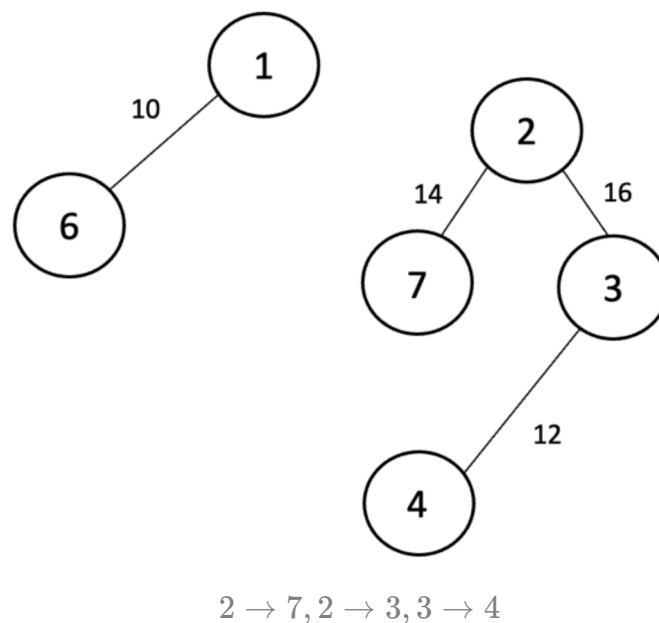
Because remember, a tree cannot contain cycles. So every time we attempt to add a new edge, we must ask: ***"Will this create a loop?"***. If yes → reject the edge. If no → include it in the MST.

Applying Kruskal's Algorithm to the Graph

Let's walk through the graph step by step.

- The smallest edge in the graph is: $1 \rightarrow 6 = 10$. So we include it in the MST.
- The second smallest edge is: $3 \rightarrow 4 = 12$. No cycle is formed, so we include it.
- The third smallest edge is: $2 \rightarrow 7 = 14$. Again, no cycle exists, so we include it.
- The fourth smallest edge is: $2 \rightarrow 3 = 16$. This edge connects the component containing node 2 and node 7 with the component containing node 3 and node 4. Still no cycle is formed. So we include it.

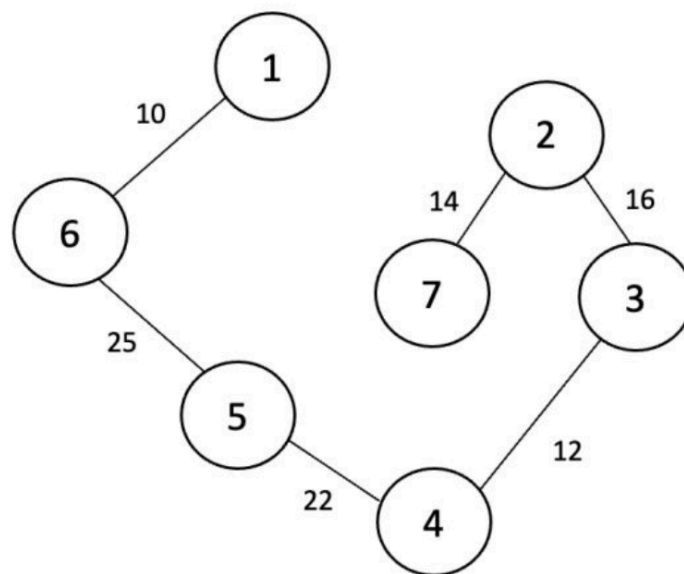
At this point, the connected structure becomes:



- The next smallest edge is: $7 \rightarrow 4 = 18$. Now we must be careful. Because node 7 is already connected to node 4 indirectly: $7 \rightarrow 2 \rightarrow 3 \rightarrow 4$. So if we add: $7 \rightarrow 4$, we create a cycle. Therefore, we reject this edge.
- Continue with the next smallest edge: $5 \rightarrow 4 = 22$. This does not create a cycle. So we include it.

- Next comes: $5 \rightarrow 7 = 24$. But node 5 is already connected to node 7 through: $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 7$. Adding this edge creates another cycle. Therefore, we reject this edge.
- Next edge: $5 \rightarrow 6 = 25$. This successfully connects the remaining disconnected components. So we include it. Finally: $1 \rightarrow 2 = 28$. But adding this edge creates a cycle because nodes 1 and 2 are already connected through the existing tree. Therefore, we reject this edge.

Final Minimum Spanning Tree:



The selected edges are:

- $1 \rightarrow 6 = 10$
- $3 \rightarrow 4 = 12$
- $2 \rightarrow 7 = 14$
- $2 \rightarrow 3 = 16$
- $5 \rightarrow 4 = 22$
- $5 \rightarrow 6 = 25$

Now let's compute the total cost:

$$10 + 12 + 14 + 16 + 22 + 25 = 99$$

So the total weight of the Minimum Spanning Tree is: 99

Time Complexity of Kruskal's Algorithm

The major operation in Kruskal's Algorithm is sorting the edges. Suppose: E = number of edges; N = number of vertices. A Minimum Spanning Tree always contains: $N - 1$ edges.

- A basic implementation of Kruskal's Algorithm may take: $O(N \times E)$ or approximately: $O(N^2)$

However, modern implementations improve this significantly using efficient sorting algorithms (Merge Sort, Quick Sort) or Union-Find (Disjoint Set) data structures for fast cycle detection

This improves the complexity to: $O(E \log E)$, which is also commonly written as: $O(E \log N)$

- The space complexity is: $O(E)$ since we mainly store the graph edges.

Advantages of Kruskal's Algorithm

- Simple and intuitive to understand
- Works very well for sparse graphs
- Efficient when combined with Union-Find
- Handles disconnected components naturally
- Focuses on edges directly rather than expanding from a starting vertex

Disadvantages of Kruskal's Algorithm

However, Kruskal's Algorithm also has limitations:

- Because it requires sorting all edges first, the sorting process may become expensive for extremely large graphs.
- In dense graphs where many edges have similar weights, Kruskal's Algorithm may also lose some of its performance advantages compared to other MST algorithms like Prim's Algorithm.

Still, despite these limitations, Kruskal's Algorithm remains one of the cleanest and most elegant examples of the greedy approach in graph theory.

3. Prim's Algorithm

Another famous greedy algorithm for finding a **Minimum Spanning Tree (MST)** is **Prim's Algorithm**, rediscovered and published by **Robert C. Prim** — which is why the algorithm carries his name today.

While Kruskal's Algorithm focuses on selecting the smallest edges globally across the graph, Prim's Algorithm thinks very differently.

Instead of looking at all edges at once, Prim's Algorithm starts from a single node, expands outward gradually, and always selects the smallest edge connected to the current tree

So rather than building many small components and merging them together like Kruskal's Algorithm, Prim's Algorithm grows one connected tree continuously.

You can think of it like this:

Kruskal builds the MST edge-by-edge globally. Prim builds the MST node-by-node locally.

The Core Idea of Prim's Algorithm

The algorithm follows a simple greedy strategy:

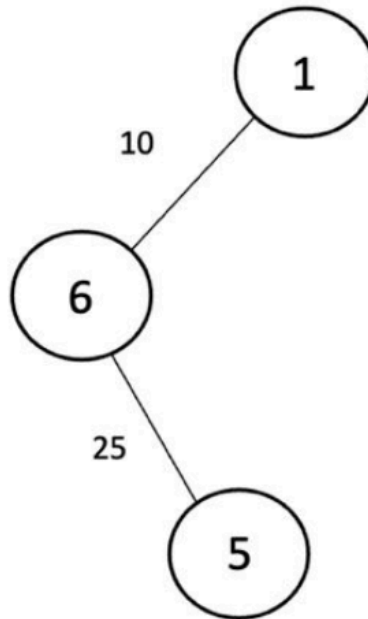
1. Start from a chosen node
2. Look at all adjacent edges connected to the current tree
3. Select the smallest edge
4. Add the new node into the tree
5. Repeat until all nodes are included

And just like before, the goal is still to minimize the total cost of the spanning tree.

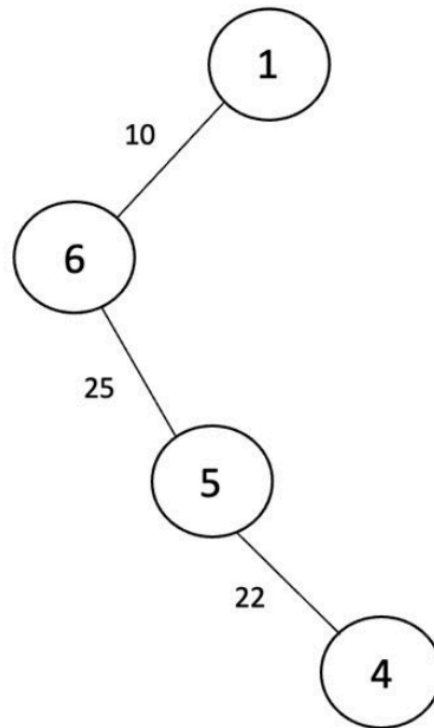
Applying Prim's Algorithm to the Graph

Let's return to the original graph and apply Prim's Algorithm step by step.

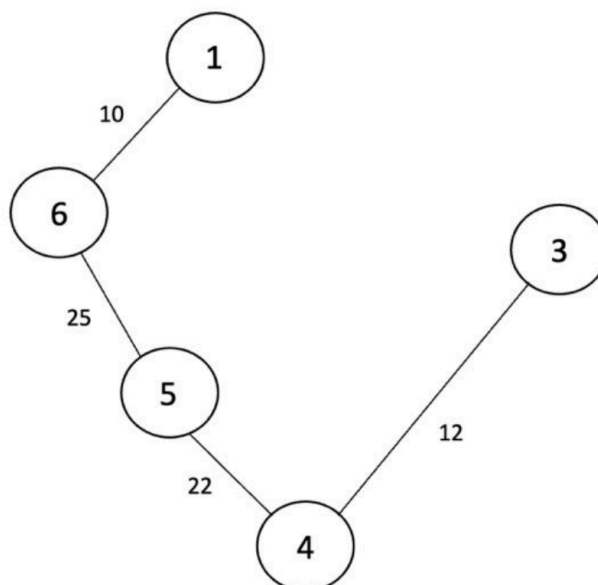
- We begin by selecting the smallest edge in the graph: $1 \rightarrow 6 = 10$. So nodes 1 and 6 become part of the MST.
- Now we examine all edges connected to the current tree. The available connections are: $1 \rightarrow 2 = 28$; $6 \rightarrow 5 = 25$.
 - Since: $25 < 28$, we select: $6 \rightarrow 5 = 25$. So node 5 is added into the MST.



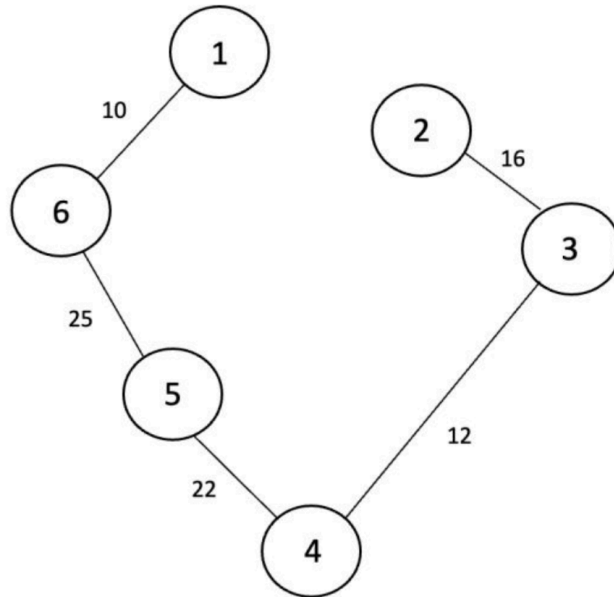
- Now the current tree contains: 1, 6, 5. The available adjacent edges are: $1 \rightarrow 2 = 28$; $5 \rightarrow 4 = 22$; $5 \rightarrow 7 = 24$.
 - The minimum edge is: $5 \rightarrow 4 = 22$. So node 4 is selected.



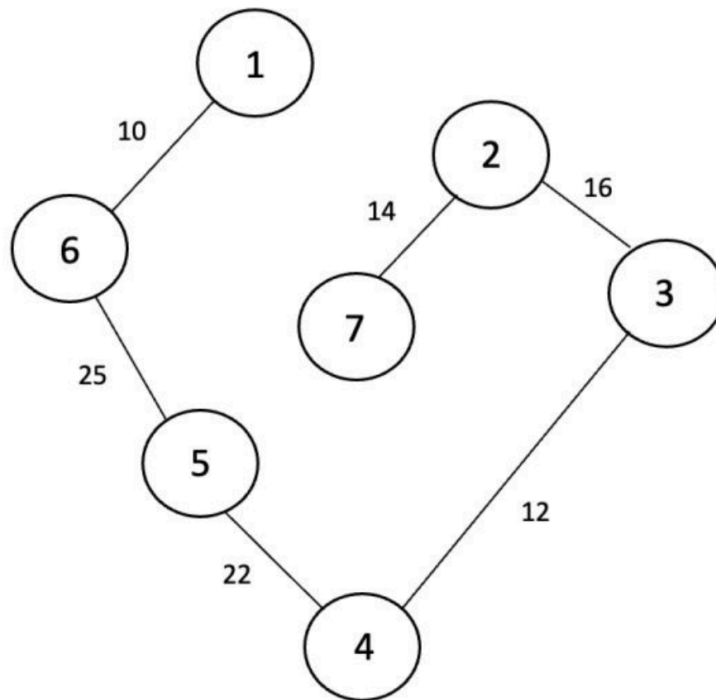
- Now the tree contains: 1, 6, 5, 4. The available edges become: $1 \rightarrow 2 = 28$; $5 \rightarrow 7 = 24$; $4 \rightarrow 3 = 12$; $4 \rightarrow 7 = 18$.
 - The minimum edge is: $4 \rightarrow 3 = 12$. So node 3 is selected.



- The current tree now contains: 1, 6, 5, 4, 3. Available edges: $1 \rightarrow 2 = 28$; $5 \rightarrow 7 = 24$; $4 \rightarrow 7 = 18$; $3 \rightarrow 2 = 16$.
 - The minimum edge is: $3 \rightarrow 2 = 16$. So node 2 is selected.



- Now the remaining unvisited node is node 7. The available edges are: $5 \rightarrow 7 = 24$; $4 \rightarrow 7 = 18$; $2 \rightarrow 7 = 14$
 - The minimum edge is: $2 \rightarrow 7 = 14$. So node 7 is selected



The selected edges are:

- $1 \rightarrow 6 = 10$
- $6 \rightarrow 5 = 25$
- $5 \rightarrow 4 = 22$
- $4 \rightarrow 3 = 12$
- $3 \rightarrow 2 = 16$
- $2 \rightarrow 7 = 14$

Now all nodes have been included. The procedure is complete.

The above image also produces the final minimum spanning tree. Interestingly, Prim's Algorithm produces the exact same MST as Kruskal's Algorithm for this graph.

Now compute the total cost:

$$10 + 25 + 22 + 12 + 16 + 14 = 99$$

So the total weight of the MST is again: 99

Time Complexity of Prim's Algorithm

Prim's Algorithm is generally more efficient than Kruskal's Algorithm for dense graphs.

- Using efficient data structures such as Priority Queues or Min-Heaps, the time complexity becomes $O(E + N \log N)$, where: E = number of edges and N = number of vertices.
- The space complexity is: $O(E + N)$, because the algorithm stores both edges and vertices during traversal.

Advantages of Prim's Algorithm

Prim's Algorithm has several strengths:

- Better performance for dense graphs
- Efficient when implemented using heaps or priority queues
- Builds the MST progressively from a single connected structure
- Often faster than Kruskal's Algorithm when many edges exist

Disadvantages of Prim's Algorithm

However, Prim's Algorithm also has limitations:

- Unlike Kruskal's Algorithm, Prim's Algorithm only works naturally on a **single connected graph**. If multiple disconnected graphs exist, each component must be handled separately.
- Another limitation appears when multiple edges share the same weight. Because Prim's Algorithm only stores currently discovered edges inside the priority queue, we have less control over which edges are ultimately selected compared to Kruskal's Algorithm, which globally sorts all edges first.
- Additionally, Prim's Algorithm is generally more difficult to implement due to the use of heaps, queues, and adjacency structures.

To conclude:

- Kruskal's Algorithm is usually easier to understand and implement. It also provides better global control over edge selection. Prim's Algorithm, however, often provides better time complexity for dense graphs.
- So neither algorithm is universally "better". The best choice depends on the graph structure, the number of edges, and the desired implementation strategy

Summary

Both Kruskal's Algorithm and Prim's Algorithm are greedy algorithms used to construct a Minimum Spanning Tree.

However, they approach the problem differently:

Kruskal's Algorithm	Prim's Algorithm
Selects edges globally	Expands from a starting node
Focuses on edges	Focuses on vertices
Easier to implement	More complex implementation
Better for sparse graphs	Better for dense graphs
Uses cycle detection heavily	Uses adjacency expansion
Time complexity: $(O(E \log E))$	Time complexity: $(O(E + N \log N))$

Even though their strategies differ, both algorithms ultimately pursue the same goal: ***Constructing a spanning tree with the minimum possible total cost.***

References

<https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>

<https://www.geeksforgeeks.org/dsa/prims-minimum-spanning-tree-mst-greedy-algo-5/>

<https://www.youtube.com/watch?v=4ZIRH0eK-qQ>